Checkers MCTS

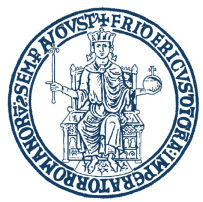
Corso di Intelligent Web

Simone Ingenito

Matricola N9000526

Università degli Studi di Napoli Federico II

Anno Accademico 2025/2026



UNIVERSITÀ DEGLI STUDI
DI NAPOLI FEDERICO II

Indice

1. Introduzione
2. Rappresentazione dello Stato
3. Algoritmo MCTS
4. Interfaccia Grafica
5. Tuning degli Iperparametri e Risultati
6. Conclusioni

Obiettivo e Vincoli del Progetto

Obiettivo: Sviluppare un Artificial Player (AP) capace di giocare alla Dama Italiana.

Requisiti della traccia:

- **Core logico:** Implementato obbligatoriamente in **C**.
- **Intelligenza Artificiale:** Basata su **Monte Carlo Tree Search (MCTS)**.
- **Configurabilità:** Limite di tempo variabile (0.2s, 1.0s, 3.0s) tramite algoritmo *Anytime*.
- **Regole:** Implementazione delle regole italiane (cattura obbligatoria, priorità di pezzo e quantità).
- **Giocabilità:** Creazione di un'interfaccia grafica per l'interazione umana.

Architettura del Sistema

Per soddisfare i requisiti garantendo massime prestazioni e un'esperienza utente moderna, l'architettura è stata divisa in tre livelli:

- **Motore Nativo (C puro):** Tutta la logica di gioco, bitboard e l'algoritmo MCTS sono scritti in C per avere il totale controllo sulla memoria (indispensabile per i limiti di tempo stretti).
- **Il Ponte (WebAssembly):** Il codice C è stato compilato tramite *Emscripten* in un modulo WASM. Questo permette di eseguire il codice nativo **direttamente nel browser** lato client a velocità quasi-nativa.
- **L'Interfaccia (React + Tailwind):** Una UI moderna e reattiva che comunica in modo asincrono con l'engine WASM per rendering grafico ed eventi utente, senza bisogno di alcun server backend.

Perché questa è la soluzione ottimale?

Nessuna Latenza di Rete

Evitando architetture Client-Server tradizionali (es. server Python in background), si annulla il *network delay*. Ogni millisecondo è riservato esclusivamente ai calcoli MCTS.

Prestazioni e Sicurezza

WebAssembly offre prestazioni molto vicine al binario nativo, ed essendo eseguito in sandbox nel browser non richiede l'installazione di librerie esterne da parte dell'utente.

Rappresentazione scacchiera: Bitboards 32-bit

Alternativa Plausibile: Usare una matrice `int board[8][8]` o un intero a 64-bit (`uint64_t`).

- **Svantaggi:** Le matrici richiedono cicli `for` lenti per iterare; un intero a 64-bit per una scacchiera in cui si usano solo 32 caselle (quelle scure) spreca il 50% della memoria e dei cicli di clock.

Scelta Effettuata: Bitboard a 32-bit (`uint32_t`)

- Mappiamo le sole 32 caselle scure su singoli bit di un intero, numerate da 0 a 31.
- Lo stato è definito al 100% da soli tre `uint32_t`: `white_pieces`, `black_pieces`, `kings` (dame).
- **Perché è migliore:** Struttura dati densa (zero sprechi). La RAM richiesta per ogni nodo è minima, permettendo alla CPU di conservare milioni di nodi nella *Cache*. Le valutazioni di posizione avvengono in parallelo su tutta la scacchiera in 1 singolo ciclo di clock (es. caselle vuote: `~(white | black)`).

Move Generation: LUTs vs Calcolo a Runtime

Alternativa Plausibile: Tradurre gli indici (0-31) in coordinate (X, Y), sommare i movimenti e usare `if (x > 0 && y < 7)` per evitare di uscire dalla scacchiera.

- **Svantaggi:** I continui controlli condizionali per i bordi (*branching*) distruggono la pipeline della CPU, risultando catastrofici in un algoritmo AnyTime che esegue milioni di cicli.

Scelta Effettuata: Pre-calcolo in 3 Array Globali (LUT)

- `MOVES[32][4]`: Maschera della casella d'atterraggio (1 passo).
- `JUMPS[32][4]`: Maschera d'atterraggio per i salti (catture a 2 passi).
- `CAPTURES[32][4]`: Maschera della casella intermedia (pezzo mangiato).

Se lo spostamento esce dai bordi della scacchiera, la matrice restituisce semplicemente il valore `0`.

Move Generation: Calcoli in $O(1)$

Grazie alle LUT, il motore non esegue mai calcoli geometrici a runtime, ma usa esclusivamente l'**Algebra Booleana**.

Validazione infallibile in un ciclo di clock (Estratto C):

```
uint32_t landing = JUMPS[sq][DIR_NW];
uint32_t victim = CAPTURES[sq][DIR_NW];

// La cattura e' valida se atterro sul vuoto E scavalco un nemico
if ((landing & empty_squares) && (victim & opponent_pieces)) {
    // Cattura autorizzata!
}
```

Perché è superiore: Non serve alcun controllo per capire se si è usciti dalla scacchiera: se il bordo viene superato, `JUMPS` restituisce `0`, rendendo falsa la condizione `landing & empty_squares`.

Aggiornamento di Stato: Bitwise XOR

Alternativa Plausibile: Usare `memcpy` per duplicare l'intero stato della scacchiera per ogni nuovo ramo dell'albero esplorato.

- **Svantaggi:** RAM e CPU verrebbero intasate di inutili allocazioni in pochi millisecondi.

Scelta Effettuata: Modifica In-Place con Operatori Bitwise
Invece di copiare la scacchiera, la modifichiamo sul posto.

1. Spostamento (Spegnere 'From', Accendere 'To'):

```
// Creiamo una maschera con il bit di partenza e di arrivo accesi
uint32_t from_to_mask = (1U << move.from) | (1U << move.to);

// Lo XOR inverte esattamente quei due bit in un colpo solo
board->white_pieces ^= from_to_mask;
```

2. Cattura (Eliminare il pezzo avversario):

```
// Usiamo l'AND combinato al NOT (~) per azzerare il bit del nemico
board->black_pieces &= ~(move.captured_men);
```

XOR: Unmake Move

Costruire l'albero MCTS significa andare "avanti e indietro nel tempo": provare una mossa (scendere) e poi annullarla per provarne un'altra (salire).

La proprietà Involutiva dello XOR: In algebra booleana, applicare lo XOR due volte con la stessa maschera annulla l'operazione ($A \oplus B \oplus B = A$).

```
// 1. Faccio la mossa per scendere nell'albero
board->white_pieces ^= from_to_mask;

// ...esplorazione dei figli...

// 2. Annulla la mossa (Unmake) riapplicando la stessa riga!
board->white_pieces ^= from_to_mask;
```

Perché è superiore: *Zero allocazioni.* L'albero esplora milioni di stati alternando in continuazione `make_move` e `unmake_move` sul singolo intero a 32-bit originale, senza **mai** creare copie della scacchiera.

Implementazione AP: Algoritmo MCTS

IL Monte Carlo Tree Search è un algoritmo di ricerca euristica che costruisce un albero decisionale in modo asimmetrico.
Si concentra sui rami più promettenti bilanciando *exploration* e *exploitation*.

Le 4 Fasi: L'algoritmo ripete questo ciclo per migliaia di volte finché non scade il tempo limite:

1. **Selection (Selezione):** Partendo dalla radice, si scende nell'albero scegliendo il figlio con il valore più alto (usando formule matematiche come UCB1 o PUCT) fino a raggiungere un nodo foglia.
2. **Expansion (Espansione):** Se la foglia non è uno stato terminale, si aggiunge un nuovo nodo figlio all'albero.
3. **Simulation / Rollout (Simulazione):** Dal nuovo nodo si gioca una partita casuale fino alla fine (Vittoria, Sconfitta o Patta).
4. **Backpropagation (Retropropagazione):** Il risultato della simulazione viene usato per aggiornare il punteggio (W) e le visite (N) di tutti i nodi attraversati.

Struttura Albero MCTS: Array vs Puntatori

Alternativa Plausibile: Nodi con array di puntatori standard (`Node**`).

- **Svantaggi:** L'allocazione a *runtime* frammenta la memoria (Cache Miss garantiti) e consuma troppa memoria (i puntatori sono a 64-bit).

Scelta Effettuata: Lista Concatenata in un Array 1D

- Ogni nodo non ha puntatori, ma usa **indici a 32-bit**:

```
int32_t first_child;
int32_t next_sibling;
```

- **Perché è superiore:** I figli non sono salvati in un array dentro al nodo, ma formano una *Lista Concatenata* all'interno del Memory Pool pre-allocato. Scorrere i figli si traduce in saltare da un indice all'altro dell'array, portando al massimo il *prefetching* hardware.

Gestione Memoria: Memory Pool vs `malloc()`

Alternativa Plausibile: Creare nodi a *runtime* chiamando `malloc()` per l'espansione e `free()` per svuotare la memoria.

- **Svantaggi:** `malloc` effettua System Calls, richiedendo cambi di contesto al Kernel. In un loop da 1 milione di iterazioni/secondo (budget di 0.2s), andrebbe a influire negativamente sulle prestazioni.

Scelta Effettuata: Pre-allocazione (Memory Pool)

- Allocazione di un singolo gigantesco array da 2 Milioni di nodi ($\sim$ 70 MB) una singola volta all'avvio.
- **Perché è superiore:** Per instanziare un nuovo nodo in fase di Expansion, si incrementa un semplice puntatore: `pool_index++` in complessità $O(1)$ assoluto. Zero frammentazione, zero *latenza* (chiamate al SO).

Fase 1: Selection (UCB1 e PUCT)

Partendo dalla radice, si scende nell'albero scegliendo il nodo figlio che massimizza una formula di *Bandit Algorithm*. Sono state implementate due politiche:

1. **Formula UCB1:**

$$UCB1 = \underbrace{\frac{W_i}{N_i}}_{\text{Exploitation}} + \underbrace{C \cdot \sqrt{\frac{\ln N_{parent}}{N_i}}}_{\text{Exploration}}$$

Bilancia lo sfruttamento delle mosse vincenti note con l'esplorazione di rami poco visitati.

2. **Formula PUCT (Predictor UCT - AlphaZero):**

$$PUCT = \frac{W_i}{N_i} + C \cdot \frac{\sqrt{N_{parent}}}{1 + N_i}$$

Differenza: Il termine di esplorazione decade più rapidamente ($1/(1+N_i)$ invece di $\sqrt{1/N_i}$), rendendo la ricerca più profonda e aggressiva.

Fase 2: Expansion

Quando la fase di selezione raggiunge un nodo che non è ancora stato completamente esplorato (nodo foglia non terminale):

- **Generazione Mosse:** Si invocano le LUT e il Move Generator C per calcolare tutte le mosse legali dello stato attuale.
- **Allocazione nel Memory Pool:** Si preleva il successivo slot libero dall'array globale ($O(1)$) per istanziare il nuovo nodo figlio.
- **Collegamento:** Il nodo viene aggiunto alla lista concatenata dei figli del padre aggiornando i puntatori interi `first_child` e `next_sibling`.
- **Inizializzazione:** Il nuovo nodo parte con: $N = 0$, $W = 0$.

Fase 3: Simulation (Rollout)

Dal nodo appena espanso si gioca una partita fino alla conclusione per ottenere una stima euristica del valore della posizione:

- **Mosse Pseudo-Casuali:** I giocatori scelgono mosse a caso tra quelle legali.
- **Risoluzione delle Patte (Cut-off):** Nella Dama due Re possono inseguirsi all'infinito. Per rispettare i limiti Anytime stringenti, abbiamo inserito:
  - Un contatore di semimosse consecutive senza catture o promozioni (regola delle 40 mosse).
  - Un tetto massimo di profondità di rollout, oltre il quale la simulazione termina dichiarando la patta (draw).
- **Esito Finale:** Restituisce: $+1$ (Vittoria), 0 (Patta), o -1 (Sconfitta).

Fase 4: Backpropagation

Il risultato ottenuto dal Rollout $(+1, 0, -1)$ deve essere comunicato a tutti gli antenati per aggiornare le conoscenze dell'albero:

- **Risalita dell'Albero:** Si ripercorre all'indietro il percorso dalla foglia espansa fino alla radice.
- **Aggiornamento Visite:** Per ogni nodo attraversato, si incrementa il conteggio visite:
$$N = N + 1$$
- **Aggiornamento Punteggio:** Si aggiorna il valore cumulato W dal punto di vista del giocatore di turno a quel livello:
$$W = W + \text{reward}$$

Al termine del tempo limite la mossa scelta dal bot sarà semplicemente quella con il maggior numero di visite N alla radice.

Gestione dei Turni: Approccio Minimax

La Dama è un gioco **a somma zero**: la vittoria di un giocatore è la sconfitta dell'avversario.

Gestione dell'alternanza dei giocatori

- **Memorizzazione Globale:** Nella Backpropagation, il punteggio W viene accumulato sempre dal punto di vista del **Bianco** ($+1.0$ vince Bianco, 0.0 vince Nero, 0.5 patta).
- **Inversione nella Selezione UCB1:** Durante la discesa, il calcolo della componente di sfruttamento (*Exploitation*) si adatta al turno:

```
float exploitation = child_score / child_visits;
if (current_player == 0) { // 0 = Turno del Nero
    exploitation = 1.0f - exploitation; // Inverte il valore!
}
```

Risultato: Il Bianco massimizza il proprio punteggio; il Nero sceglie la mossa che minimizza quello del Bianco (massimizzando il proprio).

Interfaccia Grafica e Integrazione C

Il Problema: Il motore è in C puro. Scrivere anche l'interfaccia grafica in C è piuttosto tedioso.

La Soluzione: WebAssembly (WASM) + React

- Il codice C è stato compilato tramite **Emscripten** in un binario WebAssembly (`.wasm`).
- Permette all'interfaccia React di eseguire nativamente il motore logico **direttamente nel browser**, senza latenze di rete né la necessità di un backend in C.
- Interfaccia sviluppata in React + Tailwind CSS: reattiva, intuitiva, con la possibilità di modificare in tempo reale la difficoltà.

Il Ponte WASM: Interoperabilità C-React

Il file `wasm_api.c` rappresenta il middleware che esporta lo stato del gioco e l'algoritmo al mondo Javascript.

Esportazione tramite Emscripten:

- La macro `EMSCRIPTEN_KEEPALIVE` istruisce il compilatore a non eliminare la funzione e a renderla invocabile dall'oggetto `window.Module` in JS.

Funzioni principali esportate:

- `_init_engine()`: Alloca il Memory Pool e le LUTs.
- `_get_white_pieces()`: Espone la bitboard (intero a 32-bit).
- `_play_human_move(from, to)`: Valida la mossa e aggiorna lo stato.
- `_play_bot_move(time, policy, c)`: Blocca il thread JS, esegue l'iterazione MCTS nel limite di tempo e restituisce la mossa scelta.

Gestione della Memoria (Stringhe): Le mosse legali sono restituite al JS come puntatore a stringa (`char*`). React decodifica questo indirizzo usando la funzione di utilità `UTF8ToString` integrata da Emscripten.

Gli Iperparametri del Motore Decisionale

Le prestazioni dell'Artificial Player (AP) dipendono dall'equilibrio tra **velocità di calcolo** e **qualità euristica**. Gli Iperparametri chiave sono:

1. **Costante di Esplorazione (C o α):** Pesa il termine di *Exploration* rispetto all'*Exploitation*.
   - Valori troppo alti (≥ 1.8) indirizzano l'AP verso mosse che fanno sacrificare materiale; valori troppo bassi portano l'AP a fossilizzarsi su rami che possono non essere ottimali.
2. **Limite di Cut-off del Rollout (`cutoff`):** Numero massimo di semimosse simulate prima di troncare con l'euristica di materiale.
   - Cutoff breve = simulazioni ultra-veloci (più nodi/sec); cutoff lungo = stima più profonda ma simulazioni lente.
3. **Politica di Selezione (`policy`):** La formula di discesa dell'albero: **UCB1** (esplorazione logaritmica bilanciata) vs **PUCT** (decadimento rapido in profondità stile AlphaZero).
4. **Tempo per Mossa (`time_limit`):** Budget dell'algoritmo Anytime: fissato a **0.02s** per il training massivo; scalato a **0.2s, 1.0s, 3.0s** nei preset della GUI (vincoli di traccia).

$\Longrightarrow$ Poiché C e `cutoff` sono strettamente interdipendenti il tuning è configurato su **coppie congiunte** (C, cutoff).

Tuning degli Iperparametri

Il Tuning degli iperparametri è stato formulato come un problema di **Best Arm Identification (BAI)**:

Algoritmo Successive Rejects ($T = 400$ partite):

- Spazio di ricerca formato da coppie (α, cutoff).
- Ad ogni fase, tutti i bracci attivi disputano partite a colori alternati (50% Bianco, 50% Nero).
- Il braccio con la peggior media empirica $\hat{\mu}_i$ viene eliminato definitivamente.
- I finalisti arrivano a disputare **oltre 100 partite ciascuno**, per assicuraci che la dominanza di uno sull'altro non sia frutto del "caso" ma abbia una solida base statistica.

Giustificazione Teorica del Budget (H_2): Con $T = 400$, il bound formale di Audibert et al. garantisce che il rischio d'errore $P(\text{err}) \leq \frac{K(K-1)}{2} \exp\left(-\frac{T-K}{\overline{\log}(K) H_2}\right)$ sia rigorosamente controllato.

Approfondimento: Complessità H_2 e Bound del Budget

La Formula del Limite di Errore (Audibert, Bubeck, Munos, 2010):

$$P(\text{err}) \leq \frac{K(K-1)}{2} \exp\left(-\frac{T-K}{\overline{\log}(K) \cdot H_2}\right)$$

Significato dei Termini:

- $T = 400$: Budget complessivo di partite da distribuire tra tutti i candidati.
- $K = 6$: Numero di bracci (configurazioni) in gara.
- $\overline{\log}(6) \approx 1.95$: Costante di normalizzazione armonica ($\frac{1}{2} + \sum_{i=2}^{6} \frac{1}{i}$).
- $H_2 = \max_i \left(\frac{i}{\Delta^2_{(i)}}\right)$: **Indice di difficoltà del problema**, dove $\Delta_i = \mu^* - \mu_i$ è il distacco reale tra il migliore e il braccio i. Minore è il distacco tra i bot, più alto è H_2.

Perché questa formula giustifica $T = 400$: Il termine $-\frac{T-K}{\overline{\log}(K)H_2}$ all'esponente dimostra che la probabilità di scartare per errore il braccio migliore **decade esponenzialmente** al crescere di T.

$\Longrightarrow$ Non abbiamo scelto 400 a caso: la teoria dei Banditi garantisce matematicamente che 400 partite bastano ad abbattere il rischio d'errore sui gap Δ osservati.

Esecuzione Sperimentale Successive Rejects ($T = 400$)

Allocazione progressiva delle partite ($K = 6$ bracci):

Braccio	Parametri	Match	Punti	Win Rate	Esito
Arm_1	$\alpha = \mathbf{0.8}, \text{cut} = \mathbf{30}$	**104**	**61.5**	**59.13%**	**BEST ARM**
Arm_2	$\alpha = 1.0, \text{cut} = 40$	104	60.5	58.17%	Runner-Up
Arm_3	$\alpha = 1.41, \text{cut} = 30$	70	36.5	52.14%	Eliminato
Arm_6	$\alpha = 2.2, \text{cut} = 50$	52	25.5	49.04%	Eliminato
Arm_5	$\alpha = 1.8, \text{cut} = 40$	42	19.0	45.24%	Eliminato
Arm_4	$\alpha = 1.41, \text{cut} = 50$	34	-	¡ 45%	Eliminato

Efficienza del Metodo: I bracci deboli giocano solo 34–42 partite; i due contendenti finalisti disputano 104 partite ciascuno per massimizzare la precisione statistica dove il margine è più sottile.

Convergenza Statistica e Solidità dei Risultati

Il Principio alla Base: La Legge dei Grandi Numeri

- In un gioco stocastico, un campione ridotto ($n < 20$ match) è dominato dal **rumore statistico**: una vittoria può dipendere dal caso.
- All'aumentare del numero di partite ($n > 100$), la media empirica dei punteggi si stabilizza e **converge al reale valore di forza** della configurazione.

Come l'Algoritmo Garantisce la Convergenza Finale:

- **Allocazione Progressiva:** Il Successive Rejects elimina tempestivamente i bracci deboli e concentra il budget sulle configurazioni più competitive.
- **Abbattimento della Varianza:** I due bracci finalisti arrivano a disputare **104 partite ciascuno**. L'errore casuale viene neutralizzato proprio dove i margini di distacco sono più sottili.

Implicazioni Pratiche:

- **Certezza della Decisione:** Il primato di `Arm_1` ($\alpha = 0.8, \text{cutoff} = 30$) non è una fluttuazione fortunata, ma un distacco solido e riproducibile.

Validazione Out-of-Sample: Il Torneo di Sfruttamento

Obiettivo del Test: Verificare le prestazioni fuori dal Successive Rejects per escludere qualsiasi condizionamento dell'algoritmo di selezione.

Setup del Match (200 Partite Dirette):

- **Campione:** `Arm_1` ($\alpha = 0.8, \text{cutoff} = 30$).
- **Sfidante:** `Arm_2` ($\alpha = 1.0, \text{cutoff} = 40$).
- **Simmetria Totale:** 100 partite col Bianco, 100 col Nero.

Resoconto Finale dei Match:

- **Vittorie Best Arm: 60** (30.0%)
- **Patte: 125** (62.5%)
- **Sconfitte Best Arm:** sole **15** (7.5%)
- **Punteggio Totale: 122.5 / 200 (61.25%)**
- **95% IC (Wilson):** $[54.35\%, 67.73\%]$

Approfondimento: L'Intervallo di Confidenza al 95% (Wilson)

La Formula di Wilson Score (Wilson, 1927):

$$IC_{95\%} = \frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

Applicazione ai Dati Sperimentali ($n = 200, \hat{p} = 0.6125, z = 1.96$):

- Sostituendo i valori osservati (122.5/200), otteniamo l'intervallo:

$$\mathbf{IC_{95\%}} = [\mathbf{54.35\%}, \mathbf{67.73\%}]$$

Significato Pratico e Implicazioni:

- **Interpretazione:** C'è il 95% di probabilità che la **reale forza** del Best Arm contro il Runner-Up ricada tra il 54.35% e il 67.73%.
- **Garanzia Decisiva:** Anche nello scenario pessimistico consentito dall'intervallo (54.35%), il valore è **superiore al 50% (parità)**.

$\Longrightarrow$ La superiorità di `Arm_1` è supportata da basi statistiche solide.

Validità Statistica: Dal Test d'Ipotesi al p-value

1. Formulazione dell'Ipotesi Nulla (H_0):

- Assumiamo l'ipotesi che i due bot abbiano la stessa forza ($p = 0.5$).
- Su $n = 200$ partite, il punteggio atteso dal caso puro e la deviazione standard sono:

$$\mu = n \cdot p = 200 \times 0.5 = \mathbf{100} \text{ punti}, \qquad \sigma = \sqrt{n \cdot p \cdot (1-p)} = \sqrt{50} \approx \mathbf{7.07}$$

2. Calcolo dello Z-Score (Distanza dal Caso): Il Best Arm ha ottenuto 122.5 punti (+22.5 sopra la media). Calcoliamo di quante deviazioni standard σ ci siamo allontanati dal caso:

$$z = \frac{\text{Score} - \mu}{\sigma} = \frac{122.5 - 100}{\sqrt{50}} = \frac{22.5}{7.07} = \mathbf{3.182} \quad \text{(Oltre la soglia dei } 3\sigma\text{)}$$

3. Calcolo del p-value:

$$p = P(Z \geq 3.182) = \mathbf{7.31 \times 10^{-4}} \ll 0.05 \quad (\approx \mathbf{0.07\%})$$

$\Longrightarrow$ C'è meno dello **0.07%** di probabilità che questo scarto sia frutto della fortuna. L'ipotesi nulla H_0 è formalmente respinta: la superiorità è reale con oltre il **99.9% di confidenza**.

Configurazione Finale dei Livelli di Difficoltà

Sulla base dei risultati sperimentali sono stati calibrati 4 preset nella GUI:

Livello	**Tempo**	**Policy**	**Costante** C	**Comportamento**
Facile	0.2s	UCB1	1.0	Veloce, ammette qualche imprecisione tattica
Medio	1.0s	UCB1	1.414	Solido, bilanciamento teorico standard
Difficile	1.0s	PUCT	1.414	Calcolo profondo sulle varianti principali
Gran Maestro	3.0s	UCB1	0.8	**Configurazione Ottima BAI**, quasi impeccabile

Tutti i valori rispettano i vincoli imposti dalla traccia (0.2s, 1.0s, 3.0s).

Conclusioni: Perché è la versione migliore?

Questa architettura soddisfa e supera i requisiti della traccia per 3 motivi fondamentali:

- **Velocità Estrema:** L'unione di Bitboard e Memory Pool consente all'MCTS di esplorare una quantità massiva di scenari nei limiti imposti.
- **Fedeltà alle Regole:** Le LUTs garantiscono il rispetto della variante Italiana e degli obblighi di presa.
- **Fruibilità Totale:** Grazie a WebAssembly, l'agente ad alte prestazioni in C puro gira su qualsiasi dispositivo dotato di browser, rendendo il progetto universale, sicuro e moderno.